

# Estrutura e Lógica do Código do OpenFOAM

## CFD: Curso Básico

Professor: Adriano Possebon Rosa

Departamento de Engenharia Mecânica  
Universidade de Brasília

O OpenFOAM é uma biblioteca C++ de volumes finitos, usada para montar solvers, modelos físicos, condições de contorno, malhas, utilitários e pós-processamento.

### Ideia central

Não é necessário dominar C++ para entender a lógica do código. Mas é necessário entender que cada variável importante no solver representa um objeto físico-numérico: malha, campo, fluxo, matriz, modelo ou controle do algoritmo. Tudo (ou quase tudo) é classe e objeto no OpenFOAM.

OOP: Object Oriented Programming.

## Por que o OpenFOAM usa C++?

A escolha da linguagem C++ não é acidental. Ela combina desempenho computacional com capacidade de abstração. Alguns motivos:

- permite escrever código eficiente, adequado para simulações grandes e execução em paralelo;
- permite organizar o código em classes, objetos, bibliotecas e modelos reutilizáveis;
- permite representar campos, matrizes e operadores de volumes finitos de forma próxima da notação matemática;
- permite escolher modelos em tempo de execução por meio de dicionários, sem reescrever o solver.

Nesta aula vamos olhar para o código como engenheiros usando CFD.

As perguntas principais são:

- onde estão as equações governantes?
- onde os campos  $U$ ,  $p$ ,  $T$  e  $\phi$  são criados?
- onde entram propriedades físicas, malha e condições de contorno?
- onde ocorre o loop no tempo?
- como modificar uma equação de transporte de forma controlada?

Esta não é uma aula de programação C++.

Não vamos discutir em detalhe:

- gerenciamento de memória em C++;
- templates em profundidade;
- compiladores, flags e otimização de baixo nível;
- escrita de novas classes complexas;
- implementação de modelos de turbulência ou condições de contorno novas.

Antes de modificar código, é preciso ter familiaridade com o programa e entender como ele funciona.

Primeiro, seja um usuário do OpenFOAM: rode tutoriais, altere dicionários, faça malha, monitore resíduos e valide resultados.

Modificar solver é uma etapa posterior.

Normalmente, a ordem mais segura é:

- 1 usar um tutorial oficial parecido;
- 2 alterar propriedades e condições de contorno;
- 3 usar `fvModels` ou `fvConstraints`;
- 4 modificar o código-fonte.

# Sumário

- 1 Organização do OpenFOAM
- 2 C++ básico para ler OpenFOAM
- 3 Tipos, campos e dimensões
- 4 Equações em código
- 5 Estrutura típica de um solver
- 6 foamRun e incompressibleFluid
- 7 Compilação e desenvolvimento
- 8 Modificando o icoFoam
- 9 Erros comuns e boas práticas

Uma instalação típica do OpenFOAM contém as seguintes pastas:

```
1 applications/  
2   legacy/  
3   modules/  
4   solvers/  
5   test/  
6   utilities/  
7 bin/  
8 doc/  
9 etc/  
10 platforms/  
11 src/  
12 test/  
13 thirdparty/  
14 tutorials/  
15 wmake/
```

A pasta `applications` contém programas executáveis e módulos. A pasta `src` contém bibliotecas.

# Código-fonte versus caso

É importante separar duas estruturas.

## Caso de simulação

Contém 0/, constant/ e system/. É onde o usuário define malha, campos iniciais, propriedades, esquemas e controle da solução.

## Código-fonte

Arquivos .C, .H, bibliotecas e arquivos Make. É onde estão implementados solvers, modelos, discretizações e condições de contorno.

Um caso simples contém:

```
1 case/  
2   0/  
3     U  
4     p  
5     T  
6   constant/  
7     physicalProperties  
8     momentumTransport  
9     g  
10  system/  
11    controlDict  
12    fvSchemes  
13    fvSolution  
14    blockMeshDict
```

O solver lê esses arquivos em tempo de execução.

Um solver clássico contém:

```
1 solver/  
2   solver.C  
3   createFields.H  
4   UEqn.H  
5   pEqn.H  
6   Make/  
7     files  
8     options
```

Arquivos `.H` (header files, ou arquivos de cabeçalho) são usados para declarar as classes e também são frequentemente usados para separar blocos de código e tornar o solver mais legível.

Nos arquivos `.C` está a implementação propriamente dita.

# OpenFOAM 11: mudança importante

Na versão Foundation (org), que estamos usando no nosso curso, o OpenFOAM mudou significativamente a partir da versão 11: vários solvers passaram a ser implementados como módulos.

Na minha opinião, essa mudança tornou o OpenFOAM muito melhor para usar, mas um pouco mais difícil para programar (modificar o código).

Por isso, nesta aula, vamos olhar para a estrutura do código clássico (usado na versão OpenCFD e até a versão 10 do Foundation). Essa estrutura é mais fácil de entender e modificar.

Entender essa estrutura clássica primeiro faz com que seja mais fácil entender a estrutura moderna usada no OpenFOAM 11 (e 12 e 13).

## Mudanças:

- O executável geral é **foamRun**.
- O solver específico é escolhido por `solver` em `controlDict` ou via `-solver`.
- Exemplo: `solver incompressibleFluid;`
- Muitos nomes antigos continuam como wrappers ou ficam em `legacy`.

Isso muda a organização do código em termos computacionais, mas não muda a ideia física: campos, equações, discretização e controle numérico continuam presentes (e basicamente iguais).

# Aplicação, módulo e biblioteca

## Aplicação

Programa executável: `foamRun`, `blockMesh`, `postProcess`.

## Módulo de solver

Classe carregada em tempo de execução: `incompressibleFluid`, `fluid`, `solid`, `incompressibleVoF`.

## Biblioteca

Conjunto de classes reutilizáveis: volumes finitos, malha, turbulência, transporte, termofísica, condições de contorno.

# Sumário

- 1 Organização do OpenFOAM
- 2 C++ básico para ler OpenFOAM
- 3 Tipos, campos e dimensões
- 4 Equações em código
- 5 Estrutura típica de um solver
- 6 foamRun e incompressibleFluid
- 7 Compilação e desenvolvimento
- 8 Modificando o icoFoam
- 9 Erros comuns e boas práticas

Em C++, `#include` insere o conteúdo de outro arquivo no ponto indicado.

```
1 #include "fvCFD.H"  
2 #include "createFields.H"  
3 #include "UEqn.H"  
4 #include "pEqn.H"
```

No OpenFOAM, isso é usado para organizar solvers.

### Interpretação prática

Quando o solver contém `#include "createFields.H"`, ele está lendo um bloco de código responsável por criar campos e propriedades.

Um solver tradicional tem esta estrutura:

```
1  int main(int argc, char *argv[])
2  {
3      #include "setRootCase.H"
4      #include "createTime.H"
5      #include "createMesh.H"
6      #include "createFields.H"
7
8      while (runTime.loop())
9      {
10         // resolver equacoes
11         runTime.write();
12     }
13
14     return 0;
15 }
```

# Classe e objeto

## Classe

É uma definição de tipo. Exemplo: `volVectorField` define o que é um campo vetorial volumétrico.

## Objeto

É uma instância concreta de uma classe. Exemplo: `U` é um objeto da classe `volVectorField`.

Em termos físicos: a classe é o conceito; o objeto é o campo real dentro da simulação.

## Exemplo de objeto:

```
1  volVectorField U
2  (
3      IOobject
4      (
5          "U",
6          runTime.name(),
7          mesh,
8          IOobject::MUST_READ,
9          IOobject::AUTO_WRITE
10     ),
11     mesh
12 );
```

Esse bloco cria o campo vetorial  $\mathbf{U}$ , associado à malha, lido do caso e escrito nos diretórios de tempo.

Orientação a objeto permite que o mesmo solver use diferentes modelos sem reescrever a equação.

- modelo laminar ou turbulento;
- viscosidade constante ou não-newtoniana;
- diferentes condições de contorno;
- diferentes fontes em `fvModels`;
- diferentes restrições em `fvConstraints`.

O solver chama uma interface comum. O modelo específico é escolhido por dicionário.

**Herança** significa que uma classe especializada pode reutilizar comportamento de uma classe mais geral.

### Exemplo conceitual

`incompressibleFluid` é um solver específico. Ele herda funcionalidades gerais de `fluidSolver` e de classes-base de solver.

Assim, o solver específico não precisa reimplementar tudo: controle de tempo, acesso à malha, modelos e funções comuns podem vir da infraestrutura geral.

**Polimorfismo** permite manipular objetos de diferentes classes concretas por meio de uma interface comum, fazendo com que a implementação apropriada seja selecionada em tempo de execução.

- `momentumTransport->correct()` corrige o modelo de transporte de quantidade de movimento.
- O modelo concreto pode ser laminar, RANS ou LES.
- O solver não precisa saber todos os detalhes internos de cada modelo.

Isso é essencial para um código extensível.

## Ponteiros e referências

No código do OpenFOAM aparecem frequentemente os símbolos \*, & e ->. O significado depende do contexto.

```
1  volVectorField* UPtr = &U_; // ponteiro para U_  
2  volVectorField& U = U_;     // referencia para U_  
3  
4  UPtr->correctBoundaryConditions();  
5  
6  momentumTransport->correct();  
7  
8  fvVectorMatrix& UEqn = tUEqn.ref();
```

## Interpretação prática:

- \*, em uma declaração, indica um **ponteiro**: uma variável que armazena o endereço de um objeto;
- &, em uma declaração, cria uma **referência**: outro nome para um objeto já existente;
- &U\_, em uma expressão, obtém o endereço do objeto U\_;
- -> acessa um atributo ou método por meio de um ponteiro.

Por exemplo, `UPtr->correctBoundaryConditions()` equivale a `(*UPtr).correctBoundaryConditions()`.

## Templates.

Templates permitem criar uma estrutura genérica para tipos diferentes.

```
1 GeometricField<scalar, fvPatchField, volMesh>  
2 GeometricField<vector, fvPatchField, volMesh>
```

Na prática, o OpenFOAM fornece atalhos:

```
1 volScalarField // campo escalar nas células  
2 volVectorField // campo vetorial nas células  
3 surfaceScalarField // campo escalar nas faces
```

Operator overloading.

Em C++, operadores podem ser redefinidos para objetos.

Por isso o OpenFOAM consegue escrever equações de forma parecida com a matemática:

$$\frac{\partial \mathbf{U}}{\partial t} + \nabla \cdot (\mathbf{U}\mathbf{U}) - \nabla \cdot (\nu \nabla \mathbf{U}) = -\nabla p$$

No código:

```
fvm::ddt(U) + fvm::div(phi,U) - fvm::laplacian(nu,U)
```

Essa é uma das principais ideias de projeto do OpenFOAM.

# Sumário

- 1 Organização do OpenFOAM
- 2 C++ básico para ler OpenFOAM
- 3 Tipos, campos e dimensões**
- 4 Equações em código
- 5 Estrutura típica de um solver
- 6 `foamRun` e `incompressibleFluid`
- 7 Compilação e desenvolvimento
- 8 Modificando o `icoFoam`
- 9 Erros comuns e boas práticas

## Tipos básicos usados em OpenFOAM:

| Tipo   | Interpretação       | Exemplo               |
|--------|---------------------|-----------------------|
| scalar | número real         | pressão, temperatura  |
| label  | inteiro/índice      | célula, face          |
| word   | palavra             | nome de campo         |
| vector | vetor 3D            | velocidade, gravidade |
| tensor | tensor $3 \times 3$ | gradiente, tensão     |

No OpenFOAM, esses tipos são usados em objetos maiores: campos, matrizes, malhas e modelos.

## Campos de volume e campos de superfície.

No método dos volumes finitos cell-centered:

- variáveis principais ficam nos centros das células;
- fluxos ficam nas faces;
- integrais de volume viram somas sobre faces;
- muitas operações interpolam campos dos centros das células para as faces.

Nomes importantes

`volScalarField`, `volVectorField`, `surfaceScalarField`.

## Campos comuns em escoamento incompressível.

```
1 volVectorField U;    // velocidade no centro das celulas
2 volScalarField p;    // pressao cinematica no centro das celulas
3 surfaceScalarField phi; // fluxo volumetrico nas faces
```

Para um escoamento incompressível:

$$\phi_f \approx \mathbf{U}_f \cdot \mathbf{S}_f$$

phi não é simplesmente a velocidade. É o fluxo através das faces.

Campos e condições de contorno.

Um campo em OpenFOAM não contém apenas valores internos.

Ele contém:

- valores no domínio interno;
- valores ou relações nos patches de contorno;
- dimensões físicas;
- regras de leitura e escrita;
- associação com uma malha.

Por isso, `U.correctBoundaryConditions()` é uma chamada importante.

# IObject: ponte entre arquivo e campo

```
1 IObject
2 (
3     "U",
4     runTime.name(),
5     mesh,
6     IObject::MUST_READ,
7     IObject::AUTO_WRITE
8 )
```

Esse bloco diz:

- o nome do arquivo é U;
- ele será lido no diretório de tempo atual;
- ele pertence à malha mesh;
- deve ser obrigatoriamente lido;
- será escrito nos tempos de saída.

O solver também lê propriedades de dicionários.

Exemplos:

- `constant/physicalProperties;`
- `constant/momentumTransport;`
- `system/fvSchemes;`
- `system/fvSolution;`
- `system/controlDict.`

O código não precisa ter valores numéricos fixos; eles são definidos no caso.

# Sumário

- 1 Organização do OpenFOAM
- 2 C++ básico para ler OpenFOAM
- 3 Tipos, campos e dimensões
- 4 Equações em código**
- 5 Estrutura típica de um solver
- 6 `foamRun` e `incompressibleFluid`
- 7 Compilação e desenvolvimento
- 8 Modificando o `icoFoam`
- 9 Erros comuns e boas práticas

Exemplo: icoFoam.

Para o icoFoam, a formulação básica é:

$$\nabla \cdot \mathbf{U} = 0$$

$$\frac{\partial \mathbf{U}}{\partial t} + \nabla \cdot (\mathbf{U}\mathbf{U}) - \nabla \cdot (\nu \nabla \mathbf{U}) = -\nabla p$$

Aqui,  $p$  é pressão cinemática, isto é, pressão dividida pela densidade.

No código do OpenFOAM, a equação de quantidade de movimento aparece como uma matriz de volumes finitos:

```
1 fvVectorMatrix UEqn
2 (
3     fvm::ddt(U)
4     + fvm::div(phi, U)
5     - fvm::laplacian(nu, U)
6 );
7
8 solve(UEqn == -fvc::grad(p));
```

Essa forma é uma tradução direta da equação diferencial discretizada.

## fvm versus fvc

### fvm

Operador implícito. Constrói uma matriz linear para o campo desconhecido. Exemplo: `fvm::laplacian(nu,U)`.

### fvc

Operador explícito. Calcula diretamente um campo a partir de campos conhecidos. Exemplo: `fvc::grad(p)`.

Esta distinção é central para entender estabilidade, acoplamento e linearização.

Depois da discretização, uma equação de transporte gera um sistema linear:

$$a_P \phi_P = \sum_N a_N \phi_N + b_P$$

No código, isso aparece como:

```
fvScalarMatrix, fvVectorMatrix, solve(), relax().
```

A matriz guarda coeficientes, fonte, contribuições de contorno e fluxo associado.

Exemplo.

Uma equação de temperatura poderia ser escrita como:

```
1 fvScalarMatrix TEqn
2 (
3     fvm::ddt(T)
4     + fvm::div(phi, T)
5     - fvm::laplacian(alpha, T)
6 );
7
8 TEqn.solve();
```

Isso representa:

$$\frac{\partial T}{\partial t} + \nabla \cdot (\mathbf{U}T) = \nabla \cdot (\alpha \nabla T)$$

# Sumário

- 1 Organização do OpenFOAM
- 2 C++ básico para ler OpenFOAM
- 3 Tipos, campos e dimensões
- 4 Equações em código
- 5 Estrutura típica de um solver**
- 6 `foamRun` e `incompressibleFluid`
- 7 Compilação e desenvolvimento
- 8 Modificando o `icoFoam`
- 9 Erros comuns e boas práticas

## Estrutura geral de um solver clássico:

```
1  int main(int argc, char *argv[])
2  {
3      setRootCase;
4      createTime;
5      createMesh;
6      createFields;
7
8      while (runTime.loop())
9      {
10         readOrUpdateControls;
11         solveTransportEquations;
12         solveMomentumEquation;
13         pressureVelocityCoupling;
14         correctModels;
15         writeFields;
16     }
17 }
```

Essa estrutura aparece com variações em muitos solvers.

No estilo tradicional, o loop no tempo aparece da seguinte forma:

```
1 while (runTime.loop())
2 {
3     Info<< "Time = " << runTime.userTimeName() << nl << endl;
4
5     #include "CourantNo.H"
6
7     // resolver equacoes
8
9     runTime.write();
10 }
```

O objeto runTime controla tempo inicial, final, passo de tempo e escrita.

# O que acontece em um passo de tempo?

Em um solver incompressível transiente:

- 1 atualiza número de Courant;
- 2 monta equação de quantidade de movimento;
- 3 faz predição de velocidade;
- 4 monta equação de pressão;
- 5 corrige fluxo e velocidade;
- 6 corrige condições de contorno e modelos;
- 7 escreve resultados quando necessário.

Acoplamento pressão-velocidade no código.

Para escoamento incompressível:

$$\nabla \cdot \mathbf{U} = 0$$

A pressão não vem de uma equação de estado. Ela surge como variável que força a velocidade a satisfazer continuidade.

No código, isso aparece como:

- UEqn: matriz de quantidade de movimento;
- HbyA: velocidade predita sem pressão final;
- pEqn: equação de pressão;
- phi: fluxo corrigido nas faces.

## Trecho conceitual do icoFoam:

```
1  while ( piso.correct() )
2  {
3      volScalarField rAU(1.0/UEqn.A());
4      volVectorField HbyA(constrainHbyA(rAU*UEqn.H(), U, p));
5
6      surfaceScalarField phiHbyA
7      (
8          "phiHbyA",
9          fvc::flux(HbyA) + fvc::interpolate(rAU)*fvc::ddtCorr(U, phi)
10     );
11
12     // resolver pEqn e corrigir phi, U
13 }
```

# O que são $rAU$ , $HbyA$ e $\phi HbyA$ ?

## $rAU$

Recíproco do coeficiente diagonal da matriz de momento, retornado por `UEqn.A()`; na notação algébrica usual, corresponde a  $1/a_P$ .

## $HbyA$

Parte explícita da velocidade predita, antes da correção final de pressão.

## $\phi HbyA$

Fluxo de face construído a partir da velocidade predita. Ele entra na equação de pressão.

Equação de pressão no código.

No icoFoam, a equação de pressão aparece como:

```
1 fvScalarMatrix pEqn
2 (
3     fvm::laplacian(rAU, p) == fvc::div(phiHbyA)
4 );
5
6 pEqn.setReference(pRefCell, pRefValue);
7 pEqn.solve();
8
9 phi = phiHbyA - pEqn.flux();
```

O fluxo corrigido deve satisfazer a conservação de massa discretizada.

Correção de velocidade.

Depois de resolver pressão:

```
1 U = HbyA - rAU*fvc::grad(p);  
2 U.correctBoundaryConditions();
```

Interpretação:

$$\mathbf{U} = \mathbf{H}/a_P - \frac{1}{a_P} \nabla p$$

A pressão corrige a velocidade para zerar (reduzir, na verdade) o erro da conservação de massa.

# Sumário

- 1 Organização do OpenFOAM
- 2 C++ básico para ler OpenFOAM
- 3 Tipos, campos e dimensões
- 4 Equações em código
- 5 Estrutura típica de um solver
- 6 `foamRun` e `incompressibleFluid`**
- 7 Compilação e desenvolvimento
- 8 Modificando o `icoFoam`
- 9 Erros comuns e boas práticas

No OpenFOAM 11, `incompressibleFluid` é o módulo principal para escoamento incompressível isotérmico.

Ele trata:

- regime permanente, pseudo-transiente e transiente;
- PIMPLE, com comportamento SIMPLE ou PISO dependendo dos controles;
- modelos laminares, RANS ou LES;
- modelos de fluidos não newtonianos;
- `fvModels`, `fvConstraints`, MRF e malha dinâmica.

No foamRun.C, a lógica é:

```
1 word solverName
2 (
3     runTime.controlDict().lookupOrDefault("solver", word::null)
4 );
5
6 args.optionReadIfPresent("solver", solverName);
7 solver::load(solverName);
8
9 autoPtr<solver> solverPtr(solver::New(solverName, mesh));
10 solver& solver = solverPtr();
```

O caso escolhe o solver. O executável carrega o módulo em tempo de execução.

## Estrutura resumida do loop principal do foamRun:

```
1 while (pimple.run(runTime))
2 {
3     solver.preSolve();
4     adjustDeltaT(runTime, solver);
5     runTime++;
6
7     while (pimple.loop())
8     {
9         solver.moveMesh();
10        solver.fvModels().correct();
11        solver.prePredictor();
12        solver.momentumPredictor();
13        solver.thermophysicalPredictor();
14        solver.pressureCorrector();
15        solver.postCorrector();
16    }
17
18    solver.postSolve();
19    runTime.write();
20 }
```

O foamRun não conhece os detalhes de cada solver.

Ele apenas chama métodos com nomes gerais:

- `momentumPredictor()`: monta a equação de quantidade de movimento e, quando `momentumPredictor` está habilitado, resolve o preditor de velocidade;
- `thermophysicalPredictor()`: energia ou termofísica, se existir;
- `pressureCorrector()`: pressão e continuidade;
- `postCorrector()`: corrige modelos físicos.

Cada módulo implementa esses métodos de maneira específica.

## Módulo incompressibleFluid .

### Declaração em incompressibleFluid.H:

```

1  class incompressibleFluid
2  :
3      public fluidSolver
4  {
5      protected:
6          volScalarField p_;
7          volVectorField U_;
8          surfaceScalarField phi_;
9
10         autoPtr<viscosityModel> viscosity;
11         autoPtr<incompressible::momentumTransportModel> momentumTransport;
12
13         IOMRFZoneList MRF;
14         tmp<fvVectorMatrix> tUEqn;
15     };

```

Esta é a lista dos principais objetos físicos e numéricos.

# Campos do incompressibleFluid

| Objeto            | Tipo               | Função                 |
|-------------------|--------------------|------------------------|
| p_                | volScalarField     | pressão cinemática     |
| U_                | volVectorField     | velocidade             |
| phi_              | surfaceScalarField | fluxo volumétrico      |
| viscosity         | modelo             | viscosidade cinemática |
| momentumTransport | modelo             | laminar/RANS/LES       |
| MRF               | modelo             | referência rotativa    |
| tUEqn             | matriz temporária  | equação de momento     |

## Criação da pressão p.

```
1  p_  
2  (  
3      IOobject  
4      (  
5          "p",  
6          runtime.name(),  
7          mesh,  
8          IOobject::MUST_READ,  
9          IOobject::AUTO_WRITE  
10     ),  
11     mesh  
12 )
```

O campo p é obrigatório no diretório de tempo inicial.

## Construtor: leitura de U e phi.

```
1  U_  
2  (  
3      IOobject("U", runTime.name(), mesh,  
4              IOobject::MUST_READ, IOobject::AUTO_WRITE),  
5      mesh  
6  ),  
7  
8  phi_  
9  (  
10     IOobject("phi", runTime.name(), mesh,  
11             IOobject::READ_IF_PRESENT, IOobject::AUTO_WRITE),  
12     linearInterpolate(U_) & mesh.Sf()  
13 )
```

Se phi não existir, ele é inicializado a partir de U interpolado para as faces.

Os modelos são selecionados a partir de dicionários do caso.

```
1 viscosity(viscosityModel::New(mesh)),
2
3 momentumTransport
4 (
5     incompressible::momentumTransportModel::New
6     (
7         U_,
8         phi_,
9         viscosity
10    )
11 ),
12
13 MRF(mesh)
```

## momentumPredictor.C

Equação para resolver a velocidade no primeiro passo do algoritmo:

```
1  tUEqn =
2  (
3      fvm::ddt(U) + fvm::div(phi, U)
4      + MRF.DDt(U)
5      + momentumTransport->divDevSigma(U)
6      ==
7      fvModels().source(U)
8  );
9
10 fvVectorMatrix& UEqn = tUEqn.ref();
11 UEqn.relax();
12 fvConstraints().constrain(UEqn);
13
14 if (pimple.momentumPredictor())
15 {
16     solve(UEqn == -fvc::grad(p));
17     fvConstraints().constrain(U);
18 }
```

A equação implementada é:

$$\frac{\partial \mathbf{U}}{\partial t} + \nabla \cdot (\mathbf{U}\mathbf{U}) + \text{MRF} + \text{tensões} = -\nabla p + \text{fontes}$$

No código:

- `fvm::ddt(U)`: termo transiente;
- `fvm::div(phi,U)`: convecção;
- `momentumTransport->divDevSigma(U)`: viscosidade e turbulência;
- `fvModels().source(U)`: fontes opcionais;
- `-fvc::grad(p)`: força de pressão explícita no preditor.

## Correção da pressão em correctPressure.C.

Trecho simplificado:

```
1  volScalarField rAU(1.0/UEqn.A());
2  volVectorField HbyA(constrainHbyA(rAU*UEqn.H(), U, p));
3
4  surfaceScalarField phiHbyA
5  (
6      "phiHbyA",
7      fvc::flux(HbyA)
8      + fvc::interpolate(rAU)*fvc::ddtCorr(U, phi, Uf)
9  );
10
11  fvScalarMatrix pEqn
12  (
13      fvm::laplacian(rAU(), p) == fvc::div(phiHbyA)
14  );
```

Depois da equação de pressão, vem a correção de  $\phi$  e  $U$ :

```
1  phi = phiHbyA - pEqn.flux();
2
3  continuityErrors();
4
5  p.relax();
6
7  U = HbyA - rAtU*fvc::grad(p);
8  U.correctBoundaryConditions();
9  fvConstraints().constrain(U);
```

O fluxo  $\phi$  é corrigido primeiro. Depois, o campo de velocidade  $U$  é atualizado.

# Sumário

- 1 Organização do OpenFOAM
- 2 C++ básico para ler OpenFOAM
- 3 Tipos, campos e dimensões
- 4 Equações em código
- 5 Estrutura típica de um solver
- 6 foamRun e incompressibleFluid
- 7 Compilação e desenvolvimento**
- 8 Modificando o icoFoam
- 9 Erros comuns e boas práticas

Você precisa compilar quando modifica código-fonte:

- solver .C;
- arquivos .H incluídos pelo solver;
- bibliotecas próprias;
- novas condições de contorno;
- novos modelos.

Um solver compilável tem uma pasta chamada Make.

Geralmente, a estrutura básica é esta:

```
1 solver/  
2   solver.C  
3   createFields.H  
4   Make/  
5     files  
6     options
```

Make/files informa arquivos fonte e nome do executável.

Make/options informa onde estão os headers e quais bibliotecas serão linkadas.

## Exemplo de Make/files:

```
1 icoBoussinesqFoam.C
2
3 EXE = $(FOAM_USER_APPBIN)/icoBoussinesqFoam
```

- primeira linha: arquivo fonte principal;
- EXE: caminho e nome do executável final;
- FOAM\_USER\_APPBIN: pasta de executáveis do usuário.

wmake e wclean.

Para compilar:

```
1 cd $WM_PROJECT_USER_DIR/applications/solvers/icoBoussinesqFoam
2 wmake
```

Para limpar a compilação:

```
1 wclean
2 wmake
```

wmake é o sistema de compilação do OpenFOAM.

# Prática segura de desenvolvimento

Nunca edite diretamente a instalação oficial. Muitas vezes o próprio Linux impede que você faça isso.

## Prática recomendada

Copie o solver para `$WM_PROJECT_USER_DIR` (ou qualquer outra pasta sua), mude o nome do executável e compile na área do usuário.

Assim, você preserva a instalação original e evita quebrar outros casos.

# Sumário

- 1 Organização do OpenFOAM
- 2 C++ básico para ler OpenFOAM
- 3 Tipos, campos e dimensões
- 4 Equações em código
- 5 Estrutura típica de um solver
- 6 foamRun e incompressibleFluid
- 7 Compilação e desenvolvimento
- 8 Modificando o icoFoam**
- 9 Erros comuns e boas práticas

Mãos à obra! Vamos modificar o código do OpenFOAM!

A ideia nesta seção é escolher um solver incompressível do OpenFOAM e incluir o efeito da temperatura no campo de velocidade: queremos simular o problema da convecção natural em uma cavidade, usando a aproximação de Boussinesq.

**Vamos usar o icoFoam.** O solver mais recomendado para escoamentos incompressíveis no OpenFOAM 11 é o `incompressibleFluid`, mas ele é muito complexo para uma primeira modificação.

Por isso vamos usar o `icoFoam`.

Para fins didáticos, o icoFoam é melhor porque:

- resolve apenas  $U$  e  $p$ ;
- tem o PISO escrito de forma explícita no arquivo principal;
- permite ver claramente onde adicionar uma fonte;
- tem um código mais simples e direto;
- não tem turbulência.

Para transformar o icoFoam em um solver capaz de resolver o problema de convecção natural, temos que adicionar:

- campo de temperatura  $T$ ;
- difusividade térmica  $\alpha$ ;
- equação de advecção-difusão para  $T$ ;
- gravidade  $g$ ;
- coeficiente de expansão térmica  $\beta$  e temperatura de referência  $T_{Ref}$ ;
- força de empuxo Boussinesq na equação de momento.

Equações desejadas:

$$\nabla \cdot \mathbf{U} = 0$$

$$\frac{\partial \mathbf{U}}{\partial t} + \nabla \cdot (\mathbf{U}\mathbf{U}) - \nabla \cdot (\nu \nabla \mathbf{U}) = -\nabla p - \beta(T - T_{ref})\mathbf{g}$$

$$\frac{\partial T}{\partial t} + \nabla \cdot (\mathbf{U}T) = \nabla \cdot (\alpha \nabla T)$$

Em vermelho está a parte que temos que adicionar, que não existe no icoFoam.

Considere  $\mathbf{g} = (0 \ -9.81 \ 0)$  e uma região quente com  $T > T_{ref}$ .

A força adicionada será:

$$-\beta(T - T_{ref})\mathbf{g}$$

Se  $T > T_{ref}$ , então  $-\beta(T - T_{ref})\mathbf{g}$  aponta para cima.

Isso gera movimento ascendente do fluido quente.

No OpenFOAM 11, icoFoam está na pasta

`openfoam11/applications/legacy/incompressible`

A pasta legacy é a pasta onde estão códigos de versões anteriores.

Copie toda a pasta do icoFoam para uma pasta nova sua. Iremos modificar o icoFoam copiado, nesta nova pasta, e não o icoFoam original.

O nome do nosso solver será icoBoussinesqFoam.

Dê esse nome à pasta que você copiou.

Agora vamos começar as modificações.

# Passo 1

Mude o nome do arquivo `icoFoam.C` para `icoBoussinesqFoam.C`.

Edite `Make/files`:

```
1 icoBoussinesqFoam.C
2
3 EXE = $(FOAM_USER_APPBIN)/icoBoussinesqFoam
```

Depois rode o comando abaixo, na pasta principal do solver.

```
1 wmake
```

Para verificar se deu certo, veja se o solver aparece na pasta de novos solvers do OpenFOAM.

Rode os comandos abaixo e veja se o solver `icoBoussinesqFoam` aparece.

```
1 cd $FOAM_USER_APPBIN
2 ls
```

## Passo 2

Vamos criar o campo T.

Em createFields.H, depois da criação do campo U, adicione:

```
1  volScalarField T
2  (
3      IOobject
4      (
5          "T",
6          runtime.name(),
7          mesh,
8          IOobject::MUST_READ,
9          IOobject::AUTO_WRITE
10     ),
11     mesh
12 );
```

Agora, o caso que for usar icoBoussinesqFoam precisa ter o arquivo 0/T obrigatoriamente (por causa da opção MUST\_READ).

## Passo 3

Vamos criar as propriedades térmicas.

Em createFields.H, depois de nu, adicione:

```
1  dimensionedScalar alpha
2  (
3      "alpha",
4      dimViscosity,
5      physicalProperties.lookup("alpha")
6  );
7
8  dimensionedScalar beta
9  (
10     "beta",
11     dimless/dimTemperature,
12     physicalProperties.lookup("beta")
13 );
14
15 dimensionedScalar TRef
16 (
17     "TRef",
18     dimTemperature,
19     physicalProperties.lookup("TRef")
20 );
```

Exemplo de arquivo constant/physicalProperties para um caso usando o nosso novo solver:

```
1 nu      [0 2 -1 0 0 0 0] 1e-06;  
2 alpha   [0 2 -1 0 0 0 0] 1.4e-07;  
3 beta    [0 0 0 -1 0 0 0] 2.1e-04;  
4 TRef    [0 0 0 1 0 0 0] 300;
```

nu: viscosidade cinemática. alpha: difusividade térmica. beta: coeficiente de expansão térmica. TRef: temperatura de referência.

## Passo 4

Criando a gravidade.

Adicionar em createFields.H:

```
1  Info<< "Reading g\n" << endl;  
2  uniformDimensionedVectorField g  
3  (  
4      IOobject  
5      (  
6          "g",  
7          runTime.constant(),  
8          mesh,  
9          IOobject::MUST_READ_IF_MODIFIED,  
10         IOobject::NO_WRITE  
11     )  
12 );
```

Atenção: também é preciso fazer uma mudança no arquivo icoBoussinesqFoam.C para a gravidade funcionar.

Adicione `#include "uniformDimensionedFields.H"` em icoBoussinesqFoam.C, logo acima de `using namespace Foam;` .

O trecho do icoBoussinesqFoam.C deve ficar assim:

```
1  #include "fvmDdt.H"
2  #include "fvmDiv.H"
3  #include "fvLaplacian.H"
4
5  #include "uniformDimensionedFields.H"
6
7  using namespace Foam;
```

Exemplo de arquivo constant/g para ser adicionado em um caso usando o nosso solver.

```
1 FoamFile
2 {
3     format      ascii;
4     class       uniformDimensionedVectorField;
5     location    "constant";
6     object      g;
7 }
8
9 dimensions     [0 1 -2 0 0 0 0];
10 value          (0 -9.81 0);
```

A dimensão é aceleração:  $L/T^2$ .

## Passo 5

Adicionando a equação da temperatura.

Arquivo icoBoussinesqFoam.C

No loop de tempo, logo abaixo de `#include "CourantNo.H"` e acima da equação do momento `UEqn`, adicione

```
1 fvScalarMatrix TEqn
2 (
3     fvm::ddt(T)
4     + fvm::div(phi, T)
5     - fvm::laplacian(alpha, T)
6 );
7
8 TEqn.solve();
```

Isto adiciona transporte escalar por advecção-difusão.

## Passo 6

Adicionando empuxo à equação do momento,

Está assim:

```
1 fvVectorMatrix UEqn
2 (
3     fvm::ddt(U)
4     + fvm::div(phi, U)
5     - fvm::laplacian(nu, U)
6 );
```

Vamos adicionar o empuxo. A equação de  $U$  fica assim:

```
1 fvVectorMatrix UEqn
2 (
3     fvm::ddt(U)
4     + fvm::div(phi, U)
5     - fvm::laplacian(nu, U)
6     ==
7     -beta*(T - TRef)*g
8 );
```

Esse termo que adicionamos faz com que o campo de velocidade 'sinta' o campo de temperatura.

Fizemos todas as modificações. Agora compile de novo. Rode o comando `wmake` na pasta principal do nosso solver.

**Pronto!!**

**Criamos um novo solver:** o `icoBoussinesqFoam` .

Agora vamos rodar um caso usando esse novo solver. Para isso, vamos usar o tutorial

```
openfoam11/tutorials/legacy/incompressible/icoFoam  
/cavity/cavity
```

Copie este tutorial para uma pasta sua.

Antes de modificar o caso, execute os comandos `blockMesh` e `icoFoam` para verificar o funcionamento do tutorial original e obter uma solução de referência.

# Usando o icoBoussinesqFoam

Para transformar esse caso de cavidade cisalhante em um caso de cavidade com convecção natural, precisamos fazer algumas adaptações:

- mudar o arquivo `blockMeshDict` para separar as paredes em `topWall`, `bottomWall`, `rightWall`, `leftWall` e `frontAndBack`;
- criar o arquivo `0/T`;
- modificar as condições de contorno em `0/U` e em `0/p`;
- modificar o arquivo `constant/physicalProperties` para adicionar `alpha`, `beta` e `TRef`;

- criar o arquivo `constant/g`;
- incluir `div(phi,T)` no arquivo `system/fvSchemes`;
- incluir `T` no arquivo `system/fvSolution`;
- mudar de `icoFoam` para `icoBoussinesqFoam` no arquivo `system/controlDict` .

## Exemplo de 0/T:

```
1 FoamFile
2 {
3     format      ascii;
4     class       volScalarField;
5     location    "0";
6     object      T;
7 }
8
9 dimensions     [0 0 0 1 0 0 0];
10 internalField  uniform 300;
11
12 boundaryField
13 {
14     topWall     { type zeroGradient; }
15     bottomWall  { type zeroGradient; }
16     rightWall   { type fixedValue; value uniform 300; }
17     leftWall    { type fixedValue; value uniform 310; }
18     frontAndBack { type empty; }
19 }
```

## Exemplo de modificação no fvSchemes.

```
1 ddtSchemes
2 {
3     default      Euler;
4 }
5
6 divSchemes
7 {
8     default      none;
9     div(phi,U)   Gauss linear;
10    div(phi,T)   Gauss upwind;
11 }
12
13 laplacianSchemes
14 {
15     default      Gauss linear corrected;
16 }
```

Para começar, upwind em T é mais robusto.

## Exemplo de ajuste no fvSolution.

```
1  T
2  {
3      solver      smoothSolver;
4      smoother    symGaussSeidel;
5      tolerance   1e-08;
6      relTol      0;
7  }
```

Esse bloco deve ser unido ao dicionário `solvers` já existente.

# Pronto!

Rode o código com o comando `icoBoussinesqFoam` .

O que verificar depois da modificação? Não basta compilar.

Verificar:

- se  $T$  está sendo escrito nos diretórios de tempo;
- se  $U$  muda quando a diferença de temperatura muda;
- se o resultado desaparece quando  $\text{beta}=0$ ;
- se o resultado muda de direção quando  $g$  muda de sinal;
- se o passo de tempo mantém Courant adequado.

## Perguntas para discussão:

- 1 O termo de empuxo é explícito ou implícito?
- 2 O que acontece se  $T_{Ref}$  for alterado?
- 3 O que acontece se  $g$  for invertido?
- 4 O que acontece se  $\alpha$  for muito pequeno?
- 5 O que limita o passo de tempo neste caso?

# Atenção!

## Atenção

Este solver que nós criamos é apenas para fins didáticos. Ele não substitui os solvers oficiais de convecção natural, transferência de calor ou CHT. Caso você queira resolver o problema de convecção natural com Boussinesq, use o solver `fluid`.

# Sumário

- 1 Organização do OpenFOAM
- 2 C++ básico para ler OpenFOAM
- 3 Tipos, campos e dimensões
- 4 Equações em código
- 5 Estrutura típica de um solver
- 6 `foamRun` e `incompressibleFluid`
- 7 Compilação e desenvolvimento
- 8 Modificando o `icoFoam`
- 9 Erros comuns e boas práticas

## Erros comuns ao modificar solver:

- esquecer de mudar o nome do executável em `Make/files`;
- editar o solver original da instalação;
- esquecer de criar o campo novo em `0/`;
- errar dimensões físicas;
- adicionar uma fonte com sinal errado;
- usar um caso turbulento para testar uma modificação laminar;
- tentar modificar solver modular complexo antes de entender um solver legado simples.

Quando usar `fvModels` em vez de modificar solver?

Se a alteração é uma fonte, uma restrição ou uma força já disponível, prefira `fvModels` ou `fvConstraints`.

Modificar solver faz sentido quando:

- a física não existe nos modelos disponíveis;
- você precisa resolver uma nova equação;
- a sequência do algoritmo precisa mudar;
- a modificação é parte de pesquisa em método/modelo.

Como ler um solver pela primeira vez.

Uma ordem eficiente:

- 1 leia a descrição no cabeçalho;
- 2 identifique `main()` ou a classe do solver;
- 3 localize `createFields.H` ou o construtor;
- 4 identifique campos principais;
- 5 localize equações `fvm::` e `fvc::`;
- 6 localize loops de pressão;
- 7 compare com `fvSchemes` e `fvSolution` do tutorial.

# Princípio de modificação mínima

## Boa prática

Faça uma modificação por vez e teste.

Exemplo:

- 1 copiar e compilar `icoFoam` sem alteração;
- 2 adicionar `T` e compilar;
- 3 resolver `TEqn` e compilar;
- 4 adicionar `g`, `beta`, `TRef` e compilar;
- 5 adicionar fonte de empuxo e testar.